PRD Enhancement — ROS GitHub Repository Architecture

GitHub-Backed File-System Operating System for Multi-Agent Execution

1. Purpose

This enhancement upgrades ROS into a professional GitHub-backed AI operating system. GitHub becomes the canonical engineering control plane for the ROS file-system architecture: plugins, agents, commands, skills, templates, workbooks, policies, environments, sync jobs, validation scripts, and release versions.

ROS must not live as scattered Markdown across tools. It must behave like a real software product.

```
GitHub versions the system.
Obsidian stores durable thinking.
Notion runs the operating cockpit.
Google Workspace stores final collaboration assets.
Claude Co-Work orchestrates.
Claude Code implements.
Hermes automates.
```

---

2. Strategic Principle

```
The file system is the agent.
The repository is the operating system.
The LLM is the runtime.
The tools are the execution layer.
The audit log is the memory of what happened.
```

Every ROS plugin, command, template, workbook, connector, and policy must be versioned, reviewable, testable, and promotable across environments.

---

3. GitHub Role in ROS

GitHub is responsible for:

Capability	GitHub Responsibility
Source control	Version all ROS files and agent definitions
Collaboration	Use branches, PRs, reviews, CODEOWNERS
Governance	Control changes to agent behavior and tool access
Validation	Run GitHub Actions for structure, safety, linting, and command completeness
Releases	Tag stable ROS versions
Backlog	Track plugin requests, command requests, defects, tool failures
Environments	Store environment examples and promotion policies
Audit	Preserve change history through commits, PRs, and releases

GitHub does **not** store production secrets, private client documents, raw email content, or sensitive Google Workspace data.

---

4. Canonical Repository Name

Recommended repository:

```
ros-operating-system
```

Alternative names:

```
rubin-operating-system
ros-agent-os
ros-cowork-os
```

Recommended default:

```
ros-operating-system
```

Reason: clear, scalable, professional, and not tied to one runtime.

---

5. Repository Structure

5.1 Top-Level Structure

```
ros-operating-system/
│
├── README.md
├── CHANGELOG.md
├── LICENSE.md
├── CODEOWNERS
├── CONTRIBUTING.md
├── SECURITY.md
├── .gitignore
├── .env.example
│
├── .github/
│   ├── workflows/
│   ├── ISSUE_TEMPLATE/
│   ├── PULL_REQUEST_TEMPLATE.md
│   └── dependabot.yml
│
├── 00_SYSTEM/
├── 01_ROOT_PLUGINS/
├── 02_DOMAIN_PLUGINS/
├── 03_SHARED/
├── 04_ASSETS/
├── 05_AUDIT/
├── 06_TOOL_DIAGNOSTICS/
├── 07_LLM_RUNTIME/
├── 08_GITHUB/
├── 09_ENVIRONMENTS/
├── 10_SYNC/
├── 11_NOTION/
├── 12_OBSIDIAN/
├── 13_GOOGLE_WORKSPACE/
├── 14_EVALUATION/
├── 15_AUTOMATION/
├── scripts/
├── tests/
└── docs/
```

5.2 Repository Layer Responsibilities

Folder	Purpose
`00_SYSTEM/`	Global ROS policies, principles, routing, safety, memory, context engineering
`01_ROOT_PLUGINS/`	COS and KK root plugins
`02_DOMAIN_PLUGINS/`	HV, EA, PAI, MKT, FIN domain plugins
`03_SHARED/`	Shared skills, commands, templates, workbooks, prompts, context packages
`04_ASSETS/`	Metadata and lightweight indexes for generated assets, not heavy binary storage
`05_AUDIT/`	Local audit logs, command runs, decision logs, change logs
`06_TOOL_DIAGNOSTICS/`	Tool registry, connector health, identity checks, fallback routes
`07_LLM_RUNTIME/`	Model registry, routing, handoff protocols, model permissions
`08_GITHUB/`	GitHub operating rules, branching, PRs, releases, Actions documentation
`09_ENVIRONMENTS/`	Local, sandbox, dev, stage, prod environment definitions and examples
`10_SYNC/`	Sync policies and sync lane definitions
`11_NOTION/`	Notion database schemas, templates, dashboard definitions, buttons
`12_OBSIDIAN/`	Obsidian vault mapping, publishing rules, note standards
`13_GOOGLE_WORKSPACE/`	Drive, Docs, Sheets, Slides, Gmail, Calendar usage policies
`14_EVALUATION/`	Eval checklists, scoring rubrics, feedback loops, regression tests
`15_AUTOMATION/`	Hermes cron jobs, automation specs, scheduled jobs, dry-run rules
`scripts/`	Validation, sync, scaffolding, diagnostics, export scripts
`tests/`	Structural tests, command tests, safety tests, fixture data
`docs/`	Human-readable architecture docs and implementation guides

6. Detailed Repository Structure

```
ros-operating-system/
│
├── README.md
├── CHANGELOG.md
```

```
├── CODEOWNERS
├── CONTRIBUTING.md
├── SECURITY.md
├── .gitignore
├── .env.example
│
├── .github/
│   ├── workflows/
│   │   ├── validate-ros.yml
│   │   ├── lint-markdown.yml
│   │   ├── check-plugin-structure.yml
│   │   ├── check-command-registry.yml
│   │   ├── check-template-registry.yml
│   │   ├── check-workbook-registry.yml
│   │   ├── check-tool-registry.yml
│   │   ├── check-model-registry.yml
│   │   ├── check-environment-safety.yml
│   │   ├── check-secrets.yml
│   │   ├── run-agent-evals.yml
│   │   └── release-ros.yml
│   │
│   ├── ISSUE_TEMPLATE/
│   │   ├── plugin-request.md
│   │   ├── command-request.md
│   │   ├── skill-request.md
│   │   ├── template-request.md
│   │   ├── workbook-request.md
│   │   ├── tool-failure.md
│   │   ├── environment-issue.md
│   │   ├── sync-issue.md
│   │   ├── safety-issue.md
│   │   └── bug-report.md
│   │
│   └── PULL_REQUEST_TEMPLATE.md
│
├── 00_SYSTEM/
│   ├── CLAUDE.md
│   ├── MEMORY.md
│   ├── principles.md
│   ├── routing.md
│   ├── role-hierarchy.md
│   ├── plugin-policy.md
│   ├── agent-policy.md
│   ├── skill-policy.md
│   ├── command-policy.md
│   ├── template-policy.md
```

```
│   ├── workbook-policy.md
│   ├── connector-policy.md
│   ├── identity-policy.md
│   ├── client-confidentiality-policy.md
│   ├── action-safety-policy.md
│   ├── memory-policy.md
│   ├── audit-policy.md
│   ├── context-engineering-policy.md
│   ├── context-window-budgeting.md
│   ├── feedback-loop-policy.md
│   ├── evaluation-policy.md
│   ├── markdown-instruction-principles.md
│   └── definition-of-done.md
│
├── 01_ROOT_PLUGINS/
│   ├── COS/
│   │   ├── CLAUDE.md
│   │   ├── MEMORY.md
│   │   ├── agents/
│   │   ├── skills/
│   │   ├── commands/
│   │   ├── templates/
│   │   ├── workbooks/
│   │   ├── connectors/
│   │   ├── dashboards/
│   │   ├── assets/
│   │   └── audit/
│   │
│   └── KK/
│       ├── CLAUDE.md
│       ├── MEMORY.md
│       ├── agents/
│       ├── skills/
│       ├── commands/
│       ├── templates/
│       ├── workbooks/
│       ├── connectors/
│       ├── dashboards/
│       ├── assets/
│       └── audit/
│
├── 02_DOMAIN_PLUGINS/
│   ├── HV/
│   ├── EA/
│   ├── PAI/
│   ├── MKT/
```

```
│   └── FIN/
│
├── 03_SHARED/
│   ├── skills/
│   ├── commands/
│   ├── templates/
│   ├── workbooks/
│   ├── connectors/
│   ├── prompts/
│   ├── context-packages/
│   ├── eval-checklists/
│   └── schemas/
│
├── 04_ASSETS/
│   ├── asset-index.md
│   ├── artifacts/
│   ├── reports/
│   ├── documents/
│   ├── emails/
│   ├── workbooks/
│   └── presentations/
│
├── 05_AUDIT/
│   ├── session-logs/
│   ├── command-runs/
│   ├── decision-logs/
│   ├── change-logs/
│   ├── safety-reviews/
│   └── model-handoffs/
│
├── 06_TOOL_DIAGNOSTICS/
│   ├── tool-registry.md
│   ├── connector-registry.md
│   ├── connector-health.md
│   ├── identity-checks.md
│   ├── permission-checks.md
│   ├── failure-log.md
│   ├── fallback-routes.md
│   └── diagnostic-commands.md
│
├── 07_LLM_RUNTIME/
│   ├── model-registry.md
│   ├── model-capabilities.md
│   ├── model-permissions.md
│   ├── routing-rules.md
│   ├── handoff-protocol.md
```

```
│   ├── runtime-safety-rules.md
│   ├── cost-latency-profile.md
│   └── evaluation-log.md
│
├── 08_GITHUB/
│   ├── repository-policy.md
│   ├── branching-strategy.md
│   ├── pull-request-policy.md
│   ├── issue-labels.md
│   ├── codeowners.md
│   ├── release-policy.md
│   ├── github-actions.md
│   ├── repository-security.md
│   └── repository-maintenance.md
│
├── 09_ENVIRONMENTS/
│   ├── environment-registry.md
│   ├── local.env.example
│   ├── sandbox.env.example
│   ├── dev.env.example
│   ├── stage.env.example
│   ├── prod.env.example
│   ├── secrets-policy.md
│   ├── promotion-policy.md
│   ├── environment-diagnostics.md
│   └── dry-run-policy.md
│
├── 10_SYNC/
│   ├── sync-policy.md
│   ├── source-of-truth-map.md
│   ├── github-sync.md
│   ├── obsidian-sync.md
│   ├── notion-sync.md
│   ├── google-workspace-sync.md
│   ├── hermes-cron-jobs.md
│   ├── conflict-resolution.md
│   └── sync-audit-log.md
│
├── 11_NOTION/
│   ├── notion-architecture.md
│   ├── database-schemas/
│   │   ├── plugin-registry.md
│   │   ├── agent-registry.md
│   │   ├── command-registry.md
│   │   ├── skill-registry.md
│   │   ├── template-registry.md
```

```
│   │   ├── workbook-registry.md
│   │   ├── tool-registry.md
│   │   ├── model-registry.md
│   │   ├── task-registry.md
│   │   ├── decision-log.md
│   │   ├── meeting-notes.md
│   │   ├── asset-library.md
│   │   ├── knowledge-base.md
│   │   ├── source-library.md
│   │   ├── audit-log.md
│   │   ├── feedback-log.md
│   │   └── evaluation-log.md
│   │
│   ├── templates/
│   │   ├── daily-plan.md
│   │   ├── weekly-review.md
│   │   ├── monthly-review.md
│   │   ├── hld.md
│   │   ├── adr.md
│   │   ├── meeting-brief.md
│   │   ├── decision-log.md
│   │   ├── knowledge-pattern.md
│   │   ├── deal-memo.md
│   │   ├── prd.md
│   │   ├── campaign-plan.md
│   │   ├── compliance-pack.md
│   │   └── context-package.md
│   │
│   ├── dashboards/
│   │   ├── ros-command-center.md
│   │   ├── cos-dashboard.md
│   │   ├── kk-dashboard.md
│   │   ├── hv-dashboard.md
│   │   ├── ea-dashboard.md
│   │   ├── pai-dashboard.md
│   │   ├── mkt-dashboard.md
│   │   └── fin-dashboard.md
│   │
│   ├── buttons/
│   │   ├── new-daily-plan.md
│   │   ├── new-meeting-brief.md
│   │   ├── new-decision.md
│   │   ├── new-hld.md
│   │   ├── new-deal-memo.md
│   │   ├── new-prd.md
│   │   ├── create-follow-up.md
```

```
│   │   ├── promote-to-knowledge.md
│   │   └── create-github-issue.md
│   │
│   └── synced-blocks/
│       ├── external-action-safety.md
│       ├── client-confidentiality.md
│       ├── identity-routing.md
│       ├── ai-context-block.md
│       └── definition-of-done.md
│
├── 12_OBSIDIAN/
│   ├── vault-structure.md
│   ├── publish-to-github-policy.md
│   ├── note-quality-standard.md
│   ├── knowledge-promotion-policy.md
│   ├── obsidian-to-notion-policy.md
│   └── inbox-triage-policy.md
│
├── 13_GOOGLE_WORKSPACE/
│   ├── google-workspace-policy.md
│   ├── gmail-policy.md
│   ├── calendar-policy.md
│   ├── drive-policy.md
│   ├── docs-policy.md
│   ├── sheets-policy.md
│   ├── slides-policy.md
│   ├── identity-registry.md
│   └── external-action-confirmation.md
│
├── 14_EVALUATION/
│   ├── evaluation-policy.md
│   ├── scoring-rubric.md
│   ├── feedback-loop.md
│   ├── regression-tests.md
│   ├── command-evals/
│   │   ├── cos-weekly-review.md
│   │   ├── kk-daily-plan.md
│   │   ├── hv-analyze-deal.md
│   │   ├── ea-write-hld.md
│   │   ├── ea-write-adr.md
│   │   ├── pai-create-prd.md
│   │   └── fin-compliance-pack.md
│   └── output-quality-checklists/
│       ├── executive-output.md
│       ├── architecture-output.md
│       ├── real-estate-output.md
```

```
│       ├── product-output.md
│       └── admin-output.md
│
├── 15_AUTOMATION/
│   ├── hermes-automation-policy.md
│   ├── cron-job-registry.md
│   ├── scheduled-jobs/
│   │   ├── hermes-daily-briefing.md
│   │   ├── hermes-github-backup.md
│   │   ├── hermes-notion-dashboard-refresh.md
│   │   ├── hermes-follow-up-scan.md
│   │   ├── hermes-tool-health-check.md
│   │   └── hermes-sync-audit-summary.md
│   ├── dry-runs/
│   └── runbooks/
│
├── scripts/
│   ├── validate_ros.py
│   ├── check_plugin_structure.py
│   ├── check_command_registry.py
│   ├── check_markdown_quality.py
│   ├── check_required_metadata.py
│   ├── check_no_secrets.py
│   ├── scaffold_plugin.py
│   ├── scaffold_command.py
│   ├── scaffold_template.py
│   ├── export_registry_to_notion.py
│   ├── sync_github_to_obsidian.py
│   ├── sync_obsidian_to_github.py
│   ├── generate_context_package.py
│   └── run_command_eval.py
│
├── tests/
│   ├── fixtures/
│   ├── test_plugin_structure.py
│   ├── test_command_schema.py
│   ├── test_template_schema.py
│   ├── test_workbook_schema.py
│   ├── test_safety_levels.py
│   ├── test_client_isolation.py
│   ├── test_identity_mapping.py
│   ├── test_context_package.py
│   └── test_no_secrets.py
│
└── docs/
    ├── architecture-overview.md
```

```
    ├── getting-started.md
    ├── setup-local.md
    ├── setup-obsidian.md
    ├── setup-notion.md
    ├── setup-hermes.md
    ├── setup-claude-code.md
    ├── release-process.md
    └── operating-manual.md
```

---

7. Standard Plugin Repository Structure

Every plugin must use the same structure.

```
/{PLUGIN}/
│
├── CLAUDE.md
├── MEMORY.md
├── README.md
│
├── agents/
│   └── {specialized-agent}.md
│
├── skills/
│   └── {skill-name}.md
│
├── commands/
│   └── {command-name}.md
│
├── templates/
│   └── {template-name}.md
│
├── workbooks/
│   └── {workbook-name}.md
│
├── connectors/
│   └── {connector-name}.md
│
├── dashboards/
│   └── {dashboard-name}.md
│
├── assets/
│   ├── drafts/
│   ├── final/
```

```
│   └── index.md
│
└── audit/
    ├── command-runs/
    ├── decision-logs/
    └── change-log.md
```

Minimum required files:

```
CLAUDE.md
MEMORY.md
README.md
commands/
templates/
audit/
```

Recommended full structure:

```
agents/
skills/
commands/
templates/
workbooks/
connectors/
dashboards/
assets/
audit/
```

---

8. EA Plugin Repository Structure

The EA plugin requires stricter structure because of multi-client confidentiality.

```
/02_DOMAIN_PLUGINS/EA/
│
├── CLAUDE.md
├── MEMORY.md
├── README.md
│
├── clients/
│   ├── bpost/
│   │   ├── MEMORY.md
```

```
│   │   ├── stakeholders.md
│   │   ├── meetings/
│   │   ├── hlds/
│   │   ├── adrs/
│   │   ├── risks/
│   │   ├── decisions/
│   │   └── assets/
│   │
│   ├── kbc/
│   ├── abn-amro/
│   ├── european-commission/
│   └── generic-career/
│
├── agents/
│   ├── ea-orchestrator.md
│   ├── architecture-strategy-agent.md
│   ├── solution-design-agent.md
│   ├── cloud-governance-agent.md
│   ├── security-compliance-agent.md
│   ├── platform-engineering-agent.md
│   ├── client-delivery-agent.md
│   ├── architecture-knowledge-agent.md
│   ├── architecture-document-agent.md
│   └── career-interview-agent.md
│
├── commands/
│   ├── route-request.md
│   ├── write-hld.md
│   ├── write-adr.md
│   ├── review-architecture.md
│   ├── create-risk-register.md
│   ├── prepare-client-meeting.md
│   ├── create-cloud-governance-model.md
│   ├── design-platform-engineering-model.md
│   ├── prepare-interview.md
│   └── update-client-dashboard.md
│
├── skills/
│   ├── hld-writer.md
│   ├── adr-writer.md
│   ├── architecture-review.md
│   ├── cloud-governance-review.md
│   ├── security-risk-review.md
│   ├── platform-engineering-advisor.md
│   ├── interview-prep.md
│   ├── cv-optimizer.md
```

```
│   └── client-brief-builder.md
│
├── templates/
│   ├── hld.md
│   ├── adr.md
│   ├── security-review.md
│   ├── cloud-governance-model.md
│   ├── platform-engineering-model.md
│   ├── client-meeting-brief.md
│   ├── executive-architecture-brief.md
│   ├── risk-register.md
│   └── interview-brief.md
│
├── workbooks/
│   ├── cloud-cost-model.md
│   ├── migration-wave-plan.md
│   ├── app-portfolio-assessment.md
│   ├── security-control-mapping.md
│   ├── vendor-comparison.md
│   ├── platform-capability-maturity.md
│   └── architecture-risk-scoring.md
│
├── connectors/
│   ├── notion-ea.md
│   ├── google-drive-ea.md
│   ├── gmail-bguy.md
│   ├── web-research.md
│   └── github.md
│
├── dashboards/
│   ├── client-dashboard.md
│   ├── architecture-decision-board.md
│   ├── risk-dashboard.md
│   ├── hld-adr-pipeline.md
│   └── job-pipeline.md
│
├── assets/
│   ├── hlds/
│   ├── adrs/
│   ├── reviews/
│   ├── client-briefs/
│   ├── interview-prep/
│   └── cv/
│
└── audit/
    ├── command-runs/
```

```
    ├── client-access-logs/
    ├── model-handoffs/
    ├── decision-logs/
    └── change-log.md
```

EA-specific rule:

```
No EA command may access client memory unless the client context is resolved.
No EA command may write into another client folder.
Shared EA knowledge must be sanitized before promotion.
```

---

9. File Naming Standards

9.1 Folder Names

Use lowercase with hyphens for shared and system folders:

```
context-engineering-policy.md
command-registry.md
cloud-governance-model.md
```

Plugin codes remain uppercase:

```
COS
KK
HV
EA
PAI
MKT
FIN
```

9.2 Command Files

Command files use action names without the plugin prefix:

```
/02_DOMAIN_PLUGINS/EA/commands/write-hld.md
```

Command name inside file:

```
/ea.write-hld
```

9.3 Template Files

Templates use the output asset name:

```
hld.md
adr.md
meeting-brief.md
investment-memo.md
compliance-pack.md
```

9.4 Workbook Files

Workbook files use decision-model names:

```
brrrr-model.md
dscr-ltv-model.md
cloud-cost-model.md
vendor-comparison.md
```

---

10. Required Metadata Standard

Every command, skill, template, workbook, and agent file must include a metadata block.

```
title: write-hld
entity_type: command
plugin: EA
owner_agent: solution-design-agent
status: draft
version: 0.1.0
safety_level: 2
requires_confirmation: false
requires_client_context: true
allowed_environments:
  - sandbox
  - dev
  - stage
  - prod
allowed_tools:
```

```
  - notion-ea
  - google-drive-ea
  - web-research
outputs:
  - hld
audit_required: true
last_reviewed: null
```

Metadata is mandatory because GitHub Actions must be able to validate files automatically.

---

11. Branching Strategy

```
main = stable production ROS
stage = validated pre-production ROS
dev = active integration branch
feature/plugin-{plugin-name} = new or changed plugin
feature/agent-{agent-name} = new or changed sub-agent
feature/command-{command-name} = new or changed command
feature/template-{template-name} = new or changed template
feature/workbook-{workbook-name} = new or changed workbook
fix/tool-{tool-name} = tool or connector fix
fix/safety-{topic} = safety or identity correction
experiment/{topic} = temporary prototype or research
hotfix/{issue} = urgent production correction
```

Rules:

```
No direct commits to main.
No production-impacting changes without PR.
No identity/tool/safety changes without explicit review.
No client-memory restructuring without review.
No secrets in any branch.
```

---

12. Pull Request Policy

Every PR must answer:

```
What changed?
Why is it needed?
Which plugin is affected?
```

```
Which agent is affected?
Which command, skill, template, workbook, or connector is affected?
Does this change memory behavior?
Does this change tool access?
Does this change identity routing?
Does this change safety level?
Does this affect client confidentiality?
Which environment was tested?
What validation passed?
What is the rollback plan?
```

PRs affecting the following require stricter review:

```
identity-policy.md
client-confidentiality-policy.md
action-safety-policy.md
connector-policy.md
model-permissions.md
prod.env.example
EA client structure
Gmail / Drive / Calendar actions
Level 4 or Level 5 command behavior
```

---

13. Pull Request Template

```
# ROS Pull Request

## 1. Summary
What changed and why?

## 2. Affected Area
- Plugin:
- Agent:
- Command:
- Skill:
- Template:
- Workbook:
- Connector:
- Environment:

## 3. Behavior Change
Does this change how an agent behaves?
```

```
## 4. Memory Impact
- [ ] No memory behavior changed
- [ ] Plugin memory changed
- [ ] Client memory changed
- [ ] Global memory changed

## 5. Tool / Identity Impact
- [ ] No tool access changed
- [ ] Tool access changed
- [ ] Identity routing changed
- [ ] External action behavior changed

## 6. Safety Impact
Safety level before:
Safety level after:
Confirmation required:

## 7. Client Confidentiality Impact
- [ ] No client-specific impact
- [ ] Affects EA client folders
- [ ] Affects shared knowledge promotion
- [ ] Requires confidentiality review

## 8. Validation
- [ ] Markdown lint passed
- [ ] Plugin structure check passed
- [ ] Command registry check passed
- [ ] Tool registry check passed
- [ ] Environment safety check passed
- [ ] No secrets detected
- [ ] Eval checklist updated

## 9. Test / Dry Run
Command tested:
Environment:
Result:

## 10. Rollback Plan
How do we revert safely?
```

---

14. CODEOWNERS

Recommended `CODEOWNERS`:

```
# Global ownership
* @guy

# System policies
/00_SYSTEM/ @guy

# Root plugins
/01_ROOT_PLUGINS/COS/ @guy
/01_ROOT_PLUGINS/KK/ @guy

# Domain plugins
/02_DOMAIN_PLUGINS/HV/ @guy
/02_DOMAIN_PLUGINS/EA/ @guy
/02_DOMAIN_PLUGINS/PAI/ @guy
/02_DOMAIN_PLUGINS/MKT/ @guy
/02_DOMAIN_PLUGINS/FIN/ @guy

# Sensitive architecture areas
/02_DOMAIN_PLUGINS/EA/clients/ @guy
/06_TOOL_DIAGNOSTICS/ @guy
/07_LLM_RUNTIME/ @guy
/09_ENVIRONMENTS/ @guy
/10_SYNC/ @guy
/13_GOOGLE_WORKSPACE/ @guy
/14_EVALUATION/ @guy
/15_AUTOMATION/ @guy

# GitHub control plane
/.github/ @guy
/08_GITHUB/ @guy
/scripts/ @guy
/tests/ @guy
```

Future automation contributors may be added only after clear permission boundaries exist.

---

15. GitHub Actions Validation

15.1 Required Workflows

Workflow	Purpose
`validate-ros.yml`	Runs all core validations
`lint-markdown.yml`	Checks Markdown quality and formatting

Workflow	Purpose
`check-plugin-structure.yml`	Ensures every plugin has required files/folders
`check-command-registry.yml`	Validates command metadata and naming
`check-template-registry.yml`	Validates template metadata
`check-workbook-registry.yml`	Validates workbook standards
`check-tool-registry.yml`	Ensures connector references exist
`check-model-registry.yml`	Ensures runtime references exist
`check-environment-safety.yml`	Prevents unsafe prod changes
`check-secrets.yml`	Blocks secrets and tokens
`run-agent-evals.yml`	Runs command output quality checks
`release-ros.yml`	Creates versioned ROS release

15.2 Validation Rules

GitHub Actions must fail if:

```
A plugin lacks CLAUDE.md.
A plugin lacks MEMORY.md.
A command lacks metadata.
A command lacks safety level.
A command lacks audit rule.
A command references a missing tool.
A command references a missing template.
A command references a missing workbook.
A model route references a missing model.
A file contains suspected secrets.
A prod environment file contains real credentials.
EA client folders are cross-linked incorrectly.
Markdown files exceed quality limits without justification.
External action commands lack confirmation rules.
```

---

16. Example GitHub Action — Plugin Structure

```
name: Check Plugin Structure
```

```yaml
on:
  pull_request:
    paths:
      - '01_ROOT_PLUGINS/**'
      - '02_DOMAIN_PLUGINS/**'

jobs:
  check-plugin-structure:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Check plugin structure
        run: python scripts/check_plugin_structure.py
```

---

17. Example GitHub Action — Secret Check

```yaml
name: Check Secrets

on:
  pull_request:
  push:
    branches:
      - dev
      - stage
      - main

jobs:
  check-secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run secret scan
        run: python scripts/check_no_secrets.py
```

---

18. Issue Templates

18.1 Command Request Template

```markdown
# Command Request

## Command Name
```

```
/plugin.command

## Plugin
COS / KK / HV / EA / PAI / MKT / FIN

## Problem
What repeatable workflow should this command solve?

## Required Inputs

## Expected Output

## Template Needed

## Workbook Needed

## Tools / Connectors Needed

## Safety Level

## Confirmation Required?

## Acceptance Criteria
```

18.2 Tool Failure Template

```
# Tool Failure

## Tool

## Plugin Acting

## Command Attempted

## Identity Used

## Environment
local / sandbox / dev / stage / prod

## What Failed

## Was Data Changed?

## Fallback Used
```

```
## Recommended Fix

## Safety Impact
```

18.3 Plugin Change Template

```
# Plugin Change Request

## Plugin

## Change Type
- New agent
- New command
- New skill
- New template
- New workbook
- New connector
- Boundary change
- Memory change

## Reason

## Expected Behavior

## Affected Files

## Safety / Identity Impact

## Acceptance Criteria
```

---

19. Release Strategy

ROS must use semantic versioning.

```
MAJOR.MINOR.PATCH
```

Examples:

```
v1.0.0 = First stable ROS foundation
v1.1.0 = Adds EA multi-agent system
v1.2.0 = Adds Notion context control plane
v1.3.0 = Adds Hermes cron jobs
v1.3.1 = Fixes tool diagnostic issue
```

Release notes must include:

```
Added
Changed
Deprecated
Removed
Fixed
Security
Migration notes
Known limitations
```

---

20. Environment Promotion Through GitHub

Commands and plugins move through environments via branches and PRs.

```
local → sandbox → dev → stage → prod
```

Promotion criteria:

Stage	Required Evidence
local	File structure valid
sandbox	Dry-run works with mock inputs
dev	Test connector behavior validated
stage	Real structure, restricted data, confirmation gates validated
prod	PR reviewed, diagnostics passed, rollback exists

No command can enter `prod` unless:

```
metadata is complete
safety level is defined
confirmation rule exists
tool diagnostics pass
```

```
eval checklist exists
dry-run result is logged
rollback path exists
PR is approved
```

---

21. GitHub + Notion Integration

GitHub and Notion must work together without creating chaos.

21.1 Source of Truth Split

Item	Source of Truth
Plugin definitions	GitHub
Command definitions	GitHub
Templates	GitHub, mirrored to Notion
Workbooks	GitHub, operational copy in Sheets/Notion
Tasks	Notion
Decisions	Notion, exported to GitHub when system-impacting
Issues / defects	GitHub
Dashboard status	Notion
Release versions	GitHub
Operational state	Notion

21.2 Sync Rule

```
GitHub defines what the system is.
Notion shows what the system is doing.
```

21.3 Sync Workflows

Flow	Direction	Rule
Plugin metadata	GitHub → Notion	Update Plugin Registry
Command metadata	GitHub → Notion	Update Command Registry
Template metadata	GitHub → Notion	Update Template Registry

Flow	Direction	Rule
Tool metadata	GitHub → Notion	Update Tool Registry
Notion task to issue	Notion → GitHub	Explicit command only
Feedback to issue	Notion → GitHub	Create improvement issue
Decision to repo	Notion → GitHub	Only if system-impacting

---

22. GitHub + Obsidian Integration

Obsidian is the thinking and durable knowledge layer. GitHub is the versioned system layer.

22.1 Allowed Flow

```
Obsidian note → structured knowledge → reviewed → GitHub PR
```

22.2 Not Allowed

```
Raw notes automatically becoming agent instructions.
Meeting dumps automatically entering MEMORY.md.
Client notes becoming shared patterns without sanitization.
Obsidian overwriting GitHub system files silently.
```

---

23. GitHub + Claude Code Workflow

Claude Code is the implementation contributor for repository changes.

Claude Code may:

```
Create branches.
Scaffold folders.
Create command files.
Create template files.
Write validation scripts.
Fix Markdown linting.
Update GitHub Actions.
```

```
Open PRs.
Run tests.
```

Claude Code must not:

```
Push directly to main.
Bypass PR review.
Add production secrets.
Change identity policy without review.
Change safety levels without review.
Modify EA client memory without explicit instruction.
Send emails or perform external actions.
```

---

24. GitHub + Hermes Workflow

Hermes acts as an operational runtime, especially for KK.

Hermes may:

```
Run scheduled health checks.
Create GitHub issues for failures.
Create branches for generated updates.
Update Notion operational records if authorized.
Generate sync audit summaries.
Run daily briefing workflows.
```

Hermes must not:

```
Modify production plugin behavior directly.
Merge PRs.
Send emails without confirmation.
Delete files.
Share documents.
Access EA client folders unless delegated.
```

---

25. Repository Security Rules

```
Never commit secrets.
Never commit production tokens.
```

```
Never commit raw Gmail exports.
Never commit private client documents unless explicitly approved.
Never commit passports, IDs, contracts, bank documents, or sensitive personal
data.
Never commit unrestricted Google Workspace exports.
Use .env.example only.
Use GitHub Secrets for CI/CD secrets.
Use local secret managers for runtime execution.
```

`.gitignore` must block:

```
.env
*.env
.env.local
.env.prod
secrets.*
credentials.*
token.*
*.key
*.pem
*.p12
*.json.secret
/private/
/local-only/
/client-documents/raw/
/google-export/raw/
/gmail-export/raw/
```

---

26. Repository Definition of Done

A repository change is done only when:

```
Files are in the correct folder.
Metadata is complete.
Markdown is concise and executable.
Command names follow /plugin.command.
Safety level is defined.
Confirmation rule is defined.
Tool references exist.
Template references exist.
Workbook references exist, if needed.
Client context rules are respected.
No secrets are present.
```

```
Validation passes.
Eval checklist exists for important commands.
PR explains behavior change.
Rollback path is documented.
```

---

27. MVP GitHub Implementation Plan

Phase 1 — Repository Foundation

Create:

```
README.md
CHANGELOG.md
CODEOWNERS
CONTRIBUTING.md
SECURITY.md
.gitignore
.env.example
.github/PULL_REQUEST_TEMPLATE.md
.github/ISSUE_TEMPLATE/
```

Create core folders:

```
00_SYSTEM/
01_ROOT_PLUGINS/
02_DOMAIN_PLUGINS/
03_SHARED/
06_TOOL_DIAGNOSTICS/
07_LLM_RUNTIME/
08_GITHUB/
09_ENVIRONMENTS/
10_SYNC/
11_NOTION/
14_EVALUATION/
scripts/
tests/
docs/
```

Phase 2 — Core Validation

Create scripts:

```
scripts/check_plugin_structure.py
scripts/check_command_registry.py
scripts/check_markdown_quality.py
scripts/check_required_metadata.py
scripts/check_no_secrets.py
```

Create workflows:

```
.github/workflows/validate-ros.yml
.github/workflows/lint-markdown.yml
.github/workflows/check-plugin-structure.yml
.github/workflows/check-command-registry.yml
.github/workflows/check-secrets.yml
```

Phase 3 — Plugin Scaffolding

Create plugins:

```
01_ROOT_PLUGINS/COS/
01_ROOT_PLUGINS/KK/
02_DOMAIN_PLUGINS/HV/
02_DOMAIN_PLUGINS/EA/
02_DOMAIN_PLUGINS/PAI/
02_DOMAIN_PLUGINS/MKT/
02_DOMAIN_PLUGINS/FIN/
```

Each plugin receives:

```
CLAUDE.md
MEMORY.md
README.md
commands/
templates/
audit/
```

Phase 4 — EA High-Value Build

Create EA client-safe structure:

```
02_DOMAIN_PLUGINS/EA/clients/bpost/
02_DOMAIN_PLUGINS/EA/clients/kbc/
02_DOMAIN_PLUGINS/EA/clients/abn-amro/
02_DOMAIN_PLUGINS/EA/clients/european-commission/
02_DOMAIN_PLUGINS/EA/clients/generic-career/
```

Create first EA commands:

```
route-request.md
prepare-client-meeting.md
write-hld.md
write-adr.md
review-architecture.md
prepare-interview.md
```

Phase 5 — Notion Control Plane Sync

Create:

```
11_NOTION/database-schemas/
11_NOTION/templates/
11_NOTION/dashboards/
11_NOTION/buttons/
11_NOTION/synced-blocks/
```

Add export/sync script:

```
scripts/export_registry_to_notion.py
```

---

28. Build Instruction Addendum

```
Enhance ROS with a GitHub-first repository architecture.

GitHub must be the canonical source of truth for:
```

```
- plugins
- agents
- commands
- skills
- templates
- workbooks
- policies
- environment definitions
- sync rules
- validation scripts
- release versions

Implement the repository as:
- structured folders
- mandatory metadata
- branch-based development
- PR-based review
- GitHub Actions validation
- CODEOWNERS responsibility boundaries
- issue templates
- release management
- no-secrets policy

Every plugin must follow the standard plugin structure.
EA must include strict client-separated folders.
Every command must define safety level, confirmation rule, allowed tools, output
contract, and audit rule.

Claude Code may implement repository changes through branches and PRs.
Hermes may create issues, run diagnostics, and support scheduled operational
jobs.
Neither Claude Code nor Hermes may bypass safety, identity, or PR review rules.

GitHub defines what ROS is.
Notion shows what ROS is doing.
Obsidian stores what ROS knows.
Google Workspace stores what ROS shares.
```

---

29. Final Architecture Decision

ROS will use a **GitHub-first, file-system-native architecture**.

```
The repository is the product.
The folders are the operating model.
```

```
The Markdown files are the agent instructions.
The metadata enables validation.
The PRs control behavior change.
The Actions enforce quality.
The releases define stable operating versions.
```

This is the difference between a clever prompt system and a serious AI operating system.